📖 Nivel Intermedio de Navegación en Flutter

1. **Concepto**

- En este nivel, los alumnos aprenden a manejar **rutas nombradas**, **paso de argumentos** y **control del stack de navegación**.
- Esto permite que la aplicación tenga un **mapa de pantallas centralizado**, más fácil de mantener y escalar.

---

2. **Elementos clave**

- **Rutas nombradas (`routes`)**: Se definen en `MaterialApp` y se accede a ellas con `Navigator.pushNamed`.
- **onGenerateRoute**: Permite manejar rutas dinámicas y pasar argumentos.
- **Navigator.popUntil / pushReplacement**: Controlan el stack (ejemplo: cerrar varias pantallas o reemplazar la actual).

---

3. **Teoría didáctica**

- **Ventaja de rutas nombradas**: Separan la definición de pantallas de la lógica de navegación.
- **Paso de argumentos**: Fundamental para enviar datos entre pantallas sin acoplarlas demasiado.
- **Control del stack**: Evita acumulación innecesaria de pantallas y mejora la experiencia de usuario.

💻 Ejemplo completo: Navegación intermedia con rutas nombradas y argumentos

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Navegación Intermedia',
      theme: ThemeData(primarySwatch: Colors.green),
      // Definimos rutas nombradas
      initialRoute: '/',
      routes: {
        '/': (context) => const HomeScreen(),
        '/detail': (context) => const DetailScreen(),
      },
```

```
      // Manejo de rutas dinámicas con argumentos
      onGenerateRoute: (settings) {
        if (settings.name == '/detail') {
          final args = settings.arguments as String;
          return MaterialPageRoute(
            builder: (_) => DetailScreen(data: args),
          );
        }
        return null;
      },
    );
  }
}

// ---------------- Pantalla principal ----------------
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pantalla Principal')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Ir a Detalle con argumento'),
          onPressed: () {
            Navigator.pushNamed(
              context,
              '/detail',
              arguments: 'Mensaje enviado desde Home',
            );
          },
        ),
      ),
    );
  }
}

// ---------------- Pantalla de detalle ----------------
class DetailScreen extends StatelessWidget {
  final String? data;

  const DetailScreen({super.key, this.data});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pantalla de Detalle')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(data ?? 'Sin datos', style: const TextStyle(fontSize: 20)),
            const SizedBox(height: 20),
```

```
              ElevatedButton(
                child: const Text('Volver al inicio'),
                onPressed: () {
                  Navigator.popUntil(context, ModalRoute.withName('/'));
                },
              ),
            ],
          ),
        ),
      );
    }
  }
```

🎯 Resumen

1. **Definir rutas nombradas** en `MaterialApp`.
2. **Navegar con `Navigator.pushNamed`** en lugar de `MaterialPageRoute`.
3. **Pasar argumentos entre pantallas** usando `settings.arguments`.
4. **Controlar el stack de navegación** con `popUntil` y `pushReplacement`.
5. **Diseñar apps más escalables** con un mapa central de rutas.